

Vererbung

Aspekt	Beschreibung
Definition	Vererbung ist ein Mechanismus, bei dem eine Klasse (<code>SubKlasse</code>) Eigenschaften und Verhalten von einer anderen Klasse (<code>SuperKlasse</code>) übernimmt. Dadurch wird Code-Wiederverwendung ermöglicht.
Ziel	Vermeidung von Code-Duplizierung. Förderung einer hierarchischen Struktur. Ermöglichung von Polymorphismus. Verbesserung der Wartbarkeit und Erweiterbarkeit des Codes.
Umsetzung in Java	Eine Klasse erbt von einer anderen mit dem Schlüsselwort <code>extends</code> . Eine Subklasse erhält alle nicht privaten Felder und Methoden der Superklasse. Eine Subklasse kann Methoden überschreiben oder erweitern.
Schlüsselwörter	<code>extends</code> für die Vererbung einer Klasse. <code>super</code> zum Zugriff auf Konstruktoren oder Methoden der Superklasse. <code>protected</code> für vererbte Felder und Methoden, die nicht <code>private</code> sind. <code>final</code> um zu verhindern, dass eine Klasse oder Methode überschrieben wird.
Superklasse (Basisklasse)	Die Klasse, von der geerbt wird. Kann Methoden und Felder enthalten, die von der Subklasse übernommen werden. Kann Konstruktoren bereitstellen, die von der Subklasse aufgerufen werden können.
Subklasse (abgeleitete Klasse)	Die Klasse, die von einer anderen Klasse erbt. Erbt alle nicht privaten Felder und Methoden der Superklasse. Kann Methoden überschreiben (<code>@Override</code>) oder neue Methoden hinzufügen.
Zugriffsmodifikatoren in Vererbung	<code>private</code> : Nicht vererbbar. <code>protected</code> : Vererbbar innerhalb desselben Pakets und von Subklassen. <code>public</code> : Vollständig vererbbar und überall zugänglich. <code>default</code> (kein Modifikator): Vererbbar nur innerhalb des Pakets.
Konstruktoren und Vererbung	Konstruktoren werden nicht vererbt, aber die Subklasse kann den Konstruktor der Superklasse mit <code>super()</code> aufrufen. Falls kein expliziter Konstruktor in der Subklasse definiert ist, wird automatisch der parameterlose Konstruktor der Superklasse aufgerufen.
Methodenüberschreibung (Overriding)	Eine Subklasse kann eine Methode der Superklasse mit einer neuen Implementierung ersetzen. Die Methode muss denselben Methodennamen, Rückgabotyp und Parameterliste haben. Das <code>@Override</code> -Annotation sollte verwendet werden, um Fehler zu vermeiden.

Aspekt	Beschreibung
Methodenüberladung (Overloading) vs. Überschreiben (Overriding)	Overloading: Mehrere Methoden mit demselben Namen, aber unterschiedlicher Parameterliste innerhalb derselben Klasse oder in einer Superklasse/Subklasse. Overriding: Eine Methode in einer Subklasse überschreibt eine Methode mit derselben Signatur aus der Superklasse.
super-Schlüsselwort	<code>super()</code> ruft den Konstruktor der Superklasse auf. <code>super.methodName()</code> ruft eine Methode der Superklasse auf. Wird oft verwendet, um vererbtes Verhalten zu erweitern, anstatt es komplett zu ersetzen.
final in Vererbung	Eine <code>final</code> -Methode kann nicht von einer Subklasse überschrieben werden. Eine <code>final</code> -Klasse kann nicht vererbt werden (z. B. <code>public final class A {}</code>).
Mehrfachvererbung von Klassen	Java unterstützt keine Mehrfachvererbung von Klassen (eine Klasse kann nur eine direkte Superklasse haben). Lösung: Mehrere Interfaces können implementiert werden.
Vererbung von Interfaces	Eine Klasse kann mehrere Interfaces mit <code>implements</code> implementieren. Ein Interface kann von einem anderen Interface mit <code>extends</code> erben.
Abstrakte Klassen und Vererbung	Eine abstrakte Klasse (<code>abstract class</code>) kann eine Superklasse sein. Eine Subklasse muss alle <code>abstract</code> -Methoden der Superklasse implementieren oder selbst <code>abstract</code> sein.
Zusammenhang mit Polymorphismus	Vererbung ist die Basis für Polymorphismus (eine Subklasse kann als eine Instanz der Superklasse behandelt werden). Ermöglicht Methodenaufrufe auf der Superklasse, während zur Laufzeit die Subklassenimplementierung verwendet wird.
Vorteile	Code-Wiederverwendung: Gemeinsame Logik wird nur einmal in der Superklasse definiert. Erleichterte Wartung: Änderungen an der Superklasse wirken sich auf alle Subklassen aus. Erweiterbarkeit: Neue Subklassen können leicht hinzugefügt werden.
Nachteile	Erhöhte Abhängigkeiten: Änderungen in der Superklasse können ungewollte Nebeneffekte in Subklassen haben. Kann zu tiefen Vererbungshierarchien führen: Zu viele Ebenen erschweren das Verständnis und die Wartung. Keine Mehrfachvererbung von Klassen: Einschränkung im Vergleich zu Sprachen wie C++.